INDEX

Name Anagha Hegde Std ____ Sec 4A

Roll No. ____ Subject OS - LAB School/College ____

School/College Tel. No. ____ Parents Tel. No. ____

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
1.	29/2/26	Introduction programs		
2.	03/03/26	CPU scheduling Algorithm		
3.	03/03/26	FCFS		
4.		SJF		
5.	10/03/26	Priority		10/3/26
6.	17/03/26	Round Robin		10/3/26
7.	24/03/26	Multilevel queue		24/3/26
8.	31/03/26	RMS, EDF, Proportional		31/3/26
9.	21/4/26	Boundary Buffer	10	
8.	21/4/26	Dining - Philosopher	10	28/4/26
10.	28/4/26	Safety Algorithm		
11.	5/5/26	Resource ~~Allocation~~ Request		5/5/26
12.	5/5/26	Deadlock Detection.		
13.	12/5/26	Memory Allocation	10	12/5/26
14.	26/5/26	Page Replacement Algorithm	10	26/5/26

Date __/__/__
Page _____

1. Find the second smallest element in an array.

```
#include<stdio.h>
#include<limits.h>
int main(){
    int n;
    printf("Enter number of elements:");
    scanf("%d",&n);
    if(n<2){
        printf("Need at least 2 elements\n");
        return 0;
    }
    int arr[n];
    int i;
    printf("Enter elements of array:\n");
    for(i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int min=INT_MAX;
    int min2=INT_MAX;
    for(i=0;i<n;i++){
        if(arr[i]<min){
            min2=min;
            min=arr[i];
        }
        else if(arr[i]>min && arr[i]<min2){
            min2=arr[i];
        }
    }
    if(min2==INT_MAX)
        printf("No second smallest element (all
            elements may be equal)\n");
```

Date __/__/__
Page

```
else
    printf("Second smallest element is : %d\n", num);
return 0;
}
```

Output:

Enter no of elements: 3
Enter elements of the array: 1 2 3
Second smallest element is: 2

3/3/26

Date __/__/__
Page _____

2. Write a program to add elements of left diagonal in a matrix.

```
#include<stdio.h>
int main() {
    int n, i, j, sum=0;
    printf("Enter the order of square matrix:");
    scanf("%d", &n);
    int arr[n][n];
    printf("Enter elements of the matrix:\n");
    for(i=0; i<n; i++) {
        for(j=0; j<n; j++) {
            scanf("%d", &arr[i][j]);
        }
    }
    for(i=0; i<n; i++) {
        sum += arr[i][i];
    }
    printf("Sum of left diagonal = %d\n", sum);
    return 0;
}
```

output:

Enter the order of square matrix : 2
Enter elements of the matrix : 1 2 3 4
Sum of left diagonal = 5

Date __/__/__
Page

1. Write a C program to stimulate the following CPU scheduling algorithm to find turn around time and waiting time

a) FCFS

```
#include <stdio.h>
struct Process
{
  int pid, at, bt, ct, tat, wt;
};

int main()
{
  int n, i, j;
  struct Process p[20], temp;
  int time = 0;
  printf("Enter number of processes: ");
  scanf("%d", &n);
  for (i=0; i<n; i++)
  {
    p[i].pid = i+1;
    printf("Enter arrival & Burst time for P%d", i+1);
    scanf("%d%d", &p[i].at, &p[i].bt);
  }
  for (i=0; i<n-1; i++)
  {
    for (j=0; j<n-i-1; j++)
    {
      if (p[j].at > p[j+1].at)
      {
        temp = p[j];
        p[j] = p[j+1];
```

```
            p[j+1] = temp;
        }
    }
}
for (i=0; i<n; i++)
{
    if (time < p[i].at)
        time = p[i].at;
    time += p[i].bt;
    p[i].ct = time;
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for (i=0; i<n; i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);

return 0;
}
```

Output:

```
Enter number of processes: 4
Enter Arrival & Burst time for P1 : 1 3
Enter Arrival & Burst time for P2 : 4 7
Enter Arrival & Burst time for P3 : 3 6
Enter Arrival & Burst time for P4 : 8 5
```

PID	AT	BT	CT	TAT	WT
1	1	3	4	3	0
3	3	6	10	7	1
2	4	7	17	13	6
4	8	5	22	14	9

Average WT : 4
Average TAT : 9.25

Date / /
Page

b) SJF

```
#include <stdio.h>
struct Process {
    int pid, at, bt, rt, ct, wt, tat, comp;
};

int main() {
    int n, choice;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for(int i=0; i<n; i++) {
        p[i].pid = i+1;
        printf("\nProcess %d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        p[i].rt = p[i].bt;
        p[i].comp = 0;
    }
    printf("\n1. SJF Non-Preemptive");
    printf("\n2. SJF Preemptive (SRTF)");
    printf("\nEnter Choice: ");
    scanf("%d", &choice);

    int time=0, comp=0;
    float total_wt=0, total_tat=0;
    switch(choice) {
      case 1:
        while(comp<n) {
            int idx=-1;
            int min_bt=9999;
```

Date __/__/__
Page _____

```
for (int i=0; i<n; i++) {
    if (p[i].at <= time && p[i].comp==0){
        if (p[i].bt < min_bt) {
            min_bt = p[i].bt;
            idx = i;
        }
    }
}
if (idx != -1) {
    time += p[idx].bt;
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].comp = 1;
    completed++;
    total_wt += p[idx].wt;
    total_tat += p[idx].tat;
} else {
    time++;
}
}
break;
case 2:
    while(comp<n){
        int idx = -1;
        int min_rt = 9999;
        for(int i=0; i<n; i++){
            if(p[i].at <= time && p[i].rt>0){
                if(p[i].rt < min_rt){
                    min_rt = p[i].rt;
                    idx = i;
                }
    }
```

```
        if (idx != -1) {
            p[idx].rt--;
            time++;
            if (p[idx].rt == 0) {
                p[idx].ct = time;
                p[idx].tat = p[idx].ct - p[idx].at;
                p[idx].wt = p[idx].tat - p[idx].bt;
                comp++;
                total_wt += p[idx].wt;
                total_tat += p[idx].tat;
            }
        } else {
            time++;
        }
        break;
    default:
        printf("Invalid Choice!");
        return 0;
    }
    printf("\nPID\tTAT\tBT\tCT\tWT\tTAT\n");
    for (int i=0; i<n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt,
               p[i].ct, p[i].wt, p[i].tat);
    }
    printf("\nAverage wt = %.2f", total_wt/n);
    printf("\nAverage TAT = %.2f\n", total_tat/n);
    return 0;
}
```

Output:

Date __/__/__
Page _____

Enter number of processes : 4

Process 1
Arrival time : 1
Burst time : 3

Process 2
Arrival time : 4
Burst time : 7

Process 3
Arrival time : 3
Burst time : 6

Process 4
Arrival time : 8
Burst time : 5

1. SJF Non-Preemptive
2. SJF Preemptive (SRTF)
Enter choice : 1

PID	AT	BT	CT	WT	TAT
P1	1	3	4	0	3
P2	4	7	22	11	18
P3	3	6	10	1	7
P4	8	5	15	2	7

10/3/26

Average waiting time = 3.50
Average TAT = 8.75

Enter choice: 2

PID	AT	BT	CT	WT	TAT
P1	1	3	4	0	3
P2	4	7	22	11	18
P3	5	6	10	1	7
P4	8	5	15	2	7

Average WT = 3.50
Average TAT = 8.75

Comparision:
Comparing the average WT and TAT, SJF is time taken by lesser hence more efficient than FCFS

feature	FCFS	SJF
Full form	Firstcome first serve	shortest job first
Complexity	very simple	more complex
Basis	arrival time	Burst time
Waiting Time	Usually high	lowest possible
Starvation	No	Possible
Efficiency	lower	higher

c) Priority

```
#include <stdio.h>
int main(){
    int n, i, j;
    int bt[20], wt[20], tat[20], pr[20], p[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    for(i=0; i<n; i++){
        p[i]=i+1;
        printf("Enter Burst Time and Priority for P%d:", p[i]);
        scanf("%d %d", &bt[i], &pr[i]);
    }
    for(i=0; i<n-1; i++){
        for(j=i+1; j<n; j++){
            if(pr[i] > pr[j]){
                int temp;
                temp=pr[i]; pr[i]=pr[j]; pr[j]=temp;
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=p[i]; p[i]=p[j]; p[j]=temp;
            }
        }
    }
    wt[0]=0;
    for(i=1; i<n; i++)
        wt[i]=wt[i-1]+bt[i-1];
    for(i=0; i<n; i++){
        tat[i]=bt[i]+wt[i];
        avg_wt+=wt[i];
        avg_tat+=tat[i];
    }
```

```
printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
for (i=0; i<n; i++)
    printf("P%d\t%d\t%d\t%d\t%d\n",
            p[i], bt[i], pr[i], wt[i], tat[i]);
printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);
return 0;
}
```

Output :

```
Enter number of processes: 4
Enter Burst Time and Priority for P1: 2 4
Enter Burst Time and Priority for P2: 3 2
Enter Burst Time and Priority for P3: 6 1
Enter Burst Time and Priority for P4: 7 3
```

Process	BT	Priority	WT	TAT
P3	6	1	0	6
P2	3	2	6	9
P4	7	3	9	16
P1	2	4	16	18

Average waiting time = 7.75

Average Turnaround Time = 12.25

gantt chart:

| P3 | P2 | P4 | P1 |

d) Round Robin

```
#include<stdio.h>
int main()
{
    int n, i, tq, remain;
    int bt[20], rt[20], wt[20], tat[20];
    int time=0;
    float avg_wt = 0, avg_tat=0;
    printf("Enter number of processes:");
    scanf("%d", &n);
    for (i=0; i<n; i++){
        printf("Enter Burst Time for P%d :", i+1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    printf("Enter time Quantum:");
    scanf("%d", &tq);
    remain = n;
    while (remain!=0)
    {
        for(i=0; i<n; i++){
            if (rt[i]>0){
                if (rt[i]<=tq){
                    time += rt[i];
                    wt[i] = time - bt[i];
                    rt[i] = 0;
                    remain--;
                }
                else{
                    time += tq;
                    rt[i] -= tq;
                }
            }
        }
    }
}
```

```
for (i=0; i<n; i++){
    tat[i] = bt[i] + wt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}
printf("\nProcess\tBT\tWT\tTAT\n");
for(i=0; i<n; i++)
    printf("P%d\t%d\t%d\t%d\n", i+1, bt[i],
                wt[i], tat[i]);
    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
    return 0;
}
```

Output

```
Enter number of processes: 4
Enter Burst Time For P1: 4
Enter Burst Time For P2: 5
Enter Burst Time For P3: 7
Enter Burst Time For PA: 2
Enter Time Quantum: 3
```

Process	BT	WT	TAT
P1	4	8	12
P2	5	9	14
P3	7	11	18
PA	2	9	11

10/3/26

Average waiting time = 9.25
Average Turnaround time = 13.75
Gantt Chart:

P1	P2	P3	PA	P1	PA	P3	P3
0	3	6	9	11	12	14	17	18

Priority preemptive

```
#include <stdio.h>
int main()
{
    int n, i, time=0, completed=0, highest, idx;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int pid[n], at[n], bt[n], pri[n];
    int rt[n], ct[n], tat[n], wt[n];
    for(i=0; i<n; i++)
    {
        pid[i] = i+1;
        printf("\nEnter arrival Time of P%d:", i+1);
        scanf("%d", &at[i]);
        printf("Enter burst time of P%d: ");
        scanf("%d", &bt[i]);
        printf("\nEnter Priority of P%d:");
        scanf("%d", &pri[i]);
        rt[i] = bt[i];
    }
    printf("\nGantt Chart:\n");
    while(completed<n)
    {
        highest = 10000;
        idx = -1;
        for(i=0; i<n; i++)
        {
            if(at[i] <= time && rt[i] > 0 && pri[i] < highest)
            {
                highest = pri[i];
                idx = i;
            }
        }
    }
```

```
if (idx != -1)
{
    printf (" | P%d", pid[idx]);
    bt[idx]--;
    time++;
    if (bt[idx] == 0)
    {
        completed++;
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
    }
}
else
{
    printf (" | Idle ");
    time++;
}
}
printf ("|\n\n");
printf ("PID \tAT \tBT \t PRI \tCT \t TAT \t WT\n");
for (i=0; i<n; i++)
{
    printf("P%d \t%d \t%d \t%d \t%d \t%d \t%d\n",
    pid[i], at[i], bt[i], pri[i], ct[i], tat[i], wt[i]);
}
return 0;
}
```

Output:

Enter number of processes: 3
Enter Arrival Time of P1: 2
Enter Burst Time of P1: 3
Enter Priority of P1: 4
Enter Arrival Time of P2: 2
Enter Burst Time of P2: 5
Enter Priority of P2: 7

Enter Arrival Time of P3: 5
Enter Burst Time of P3: 6
Enter Priority of P3: 8

Gantt Chart:

| Idle | Idle | P1 | P1 | P2 | P2 | P2 | P2 | P3 | P3 | P3 | P3 | P3 | P3 |

PID	AT	BT	PRI	CT	TAT	WT
P1	2	3	4	5	3	0
P2	2	5	7	10	8	3
P3	5	6	8	16	11	5

10/3/26

Conclusion for Round Robin

Smaller quantum time improves responsiveness and reduces waiting time.
Larger quantum time increases waiting time.

Multilevel queue

```
#include <stdio.h>
#define MAX 100

typedef struct {
    int id, at, bt, rt, ct, wt, tat, type;
} Process;

Process p[MAX];
int n;
int sysQ[MAX], userQ[MAX];
int frontS = 0, rearS = 0;
int frontU = 0, rearU = 0;

void enqueue(int q[], int *rear, int val) {
    q[(*rear)++] = val;
}
int dequeue(int q[], int *front, int *rear) {
    if (*front == *rear) return -1;
    return q[(*front)++];
}
int main() {
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        printf("\nProcess %d\n", i);
        printf("Enter arrival time: ");
        scanf("%d", &p[i].at);
        printf("Enter burst time: ");
        scanf("%d", &p[i].bt);
        printf("Enter type (0=System, 1=User): ");
        scanf("%d", &p[i].type);
        p[i].id = i;
        p[i].rt = p[i].bt;
    }
    for (int i=0; i<n-1; i++) {
        for (int j=i+1; j<n; j++) {
            if (p[i].at > p[j].at) {
                Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    int time=0, complete=0, i=0;
    int current = -1;
    int gantt[MAX], gtime[MAX], gindex=0;
    while (complete < n) {
        while (i<n && p[i].at <= time) {
            if (p[i].type == 0)
                enqueue(sysQ, &rearS, i);
            else
                enqueue(userQ, &rearU, i);
            i++;
        }
        if (current != -1 && p[current].type == 1 &&
                !isEmpty(frontS, rearS)) {
            enqueue(userQ, &rearU, current);
            current = -1;
        }
        if (current == -1) {
            if (!isEmpty(frontS, rearS))
                current = dequeue(sysQ, &frontS, &rearS);
            else if (!isEmpty(frontU, rearU))
```

```
        current = dequeue(userQ, &frontU, &rearU);
    else {
        time++;
        continue;
    }
}
gantt[gindex] = p[current].id;
gtime[gindex] = time;
gindex++;
p[current].rt--;
time++;
if (p[current].rt == 0) {
    p[current].ct = time;
    p[current].tat = p[current].ct - p[current].at;
    p[current].wt = p[current].tat - p[current].bt;
    completed++;
    current = -1;
}
}
gtime[gindex] = time;
printf("\nGantt chart:\n");
for(int i=0; i<gindex; i++) {
    printf("P%d |", gantt[i]);
}
printf("\n0");
for(int i=0; i<gindex; i++) {
    printf(" %d", gtime[i+1]);
}
float totalWT = 0, totalTAT = 0;
printf("\n\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");
for(int i=0; i<n; i++) {
    printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id, (p[i].type == 0) ? "System" : "User",
        p[i].at, p[i].bt, p[i].ct, p[i].wt, p[i].tat);

    totalWT += p[i].wt;
    totalTAT += p[i].tat;
}
printf("\nAverage Waiting Time = %.2f", totalWT/n);
printf("\nAverage Turnaround Time = %.2f\n", totalTAT/n);
return 0;
}
```

Output:

```
Enter number of processes: 3

process 0
Enter arrival time: 1
Enter burst time: 3
Enter type (0=System, 1=User): 1

Process 1
Enter arrival time: 2
Enter burst time: 3
Enter type (0=System, 1=User): 0

Process 2
Enter arrival time: 1
Enter burst time: 4
Enter type (0=System, 1=User): 0

Gantt chart:
|P2|P2|P2|P2|P2|P2|P2|P1|P1|P1|P0|
0  1  2  3  4  5  6  7  8  9  10 11
|P0|P0|P0|
12 13 14
```

ID	Type	AT	BT	CT	WT	TAT
0	User	1	3	14	10	15
1	System	2	7	87	0	7
2	System	81	3	10	6	9

Average waiting Time : 5.33
Average Turnaround Time : 9.67

10/10
24/3/26

Write a program to simulate Real-Time CPU scheduling algorithms
a) Rate Monotonic
b) Earliest - deadline first
c) Proportional scheduling

```
#include<stdio.h>
#include<math.h>
#define MAX 20

typedef struct {
    int id;
    int burst;
    int deadline;
    int period, share, waiting, turnaround, completion;
} Process;

void edf(Process p[], int n) {
    Process temp;
    float util = 0;
    for (int i=0; i<n; i++)
        util += (float) p[i].burst / p[i].deadline;
    printf("\nEDF Utilization = %.2f", util);
    printf("util <= 1 ? "\nEDF: SCHEDULABLE\n"
                      : "\nEDF: NON-SCHEDULABLE\n");
    for (int i=0; i<n-1; i++) {
        for (int j=i+1; j<n; j++) {
            if (p[i].deadline > p[j].deadline) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
```

```
    int time = 0;
    printf("Earliest deadline first (EDF)\n");
    printf("ID\tBT\tDeadline\tCT\tWT\tTAT\n");
    for (int i=0; i<n; i++) {
        p[i].waiting = time;
        time += p[i].burst;
        p[i].completion = time;
        p[i].turnaround = p[i].completion;
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].id, p[i].burst, p[i].deadline, p[i].completion,
            p[i].waiting, p[i].turnaround);
    }
}

float calculate_utilization(Process p[], int n, int use_period) {
    float utilization = 0;
    for (int i=0; i<n; i++) {
        utilization += (float) p[i].burst / (use_period
            ? p[i].period : p[i].deadline);
    }
    return utilization;
}

void rms(Process p[], int n) {
    float u = calculate_utilization(p, n, 1);
    float limit = n * (pow(2.0, 1.0/n) - 1);
    printf("\n Rate Monotonic Scheduling (RMS)");
    printf("\n Total Utilization: %.2f [Limit: %.2f]", u, limit);
    if (u > 1.0) {
        printf("\n Result: Definitely not schedulable (Utilization > 1)\n");
        return;
    } else if (u > limit) {
        printf("\n Result: potentially not schedulable");
    } else {
        printf("\n Result: Schedulable");
    }
    process temp;
    for (int i=0; i<n-1; i++) {
        for (int j=i+1; j<n; j++) {
            if (p[i].period > p[j].period) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    int time = 0;
    printf("\nPID\tBT\tPED\tCT\tWT\tTAT\n");
    for (int i=0; i<n; i++) {
        p[i].waiting = time;
        time += p[i].burst;
        p[i].completion = time;
        p[i].turnaround = p[i].completion;
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id,
            p[i].burst, p[i].period, p[i].completion,
            p[i].waiting, p[i].turnaround);
    }
}

void proportional(process p[], int n) {
    int total_shares = 0;
    for(int i=0; i<n; i++) total_shares +=
        p[i].share;
```

```
    if (total_shares > 100) {
        printf("Result : Overload");
    } else {
        printf("Result : Schedulable");
    }
    int time = 0;
    printf("ID\tBT\tShare\tCT\tWT\tTAT\n");
    for (int i=0; i<n; i++) {
        p[i].waiting = time;
        time += p[i].burst;
        p[i].completion = time;
        p[i].turnaround = p[i].completion;
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].id, p[i].burst, p[i].share, p[i].
            completion, p[i].waiting, p[i].turnaround);
    }
}
int main() {
    int n;
    Process p[MAX], p1[MAX], p2[MAX], p3[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        p[i].id = i+1;
        printf("\nProcess %d (Burst, Deadline, Period,
                Share): ", i+1);
        scanf("%d %d %d %d", &p[i].burst, &p[i].
            deadline, &p[i].period, &p[i].share);
        p1[i] = p2[i] = p3[i] = p[i];
    }
    rms(p2, n);
    edf(p1, &n);
    proportional(p3, n);
    return 0;
}
```

Output:

```
Enter number of processes: 3
Process 1 (Burst, Deadline, Period, Share): 3 10 6 2
Process 2 (Burst, Deadline, Period, Share): 2 5 4 1
Process 3 (Burst, Deadline, Period, Share): 1 8 3 7
```

Rate Monotonic Scheduling (RMS)
Total utilization 1.33 (Limit: 0.78)
Result: Definitely not schedulable (utilization > 1)

Earliest Deadline first (EDF)
Total Utilization: 0.83
Result: Schedulable

ID	BT	DL	CT	WT	TAT
2	2	5	2	0	2
3	1	8	3	2	3
1	3	10	6	3	6

gantt chart:

P2	P3	P1
0	2	3	6

21/3/25

Proportional Share Scheduling
Total shares allocated: 4
Result: Schedulable

ID	BT	Share	CT	WT	TAT
1	3	2	3	0	3
2	2	1	5	3	5
3	1	1	6	5	6

P1	P2	P3

1. The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized $S_0 = 1$, $S_1 = 0$, $S_2 = 0$. Find out how many times Process P_0 will print "0". Assume the order of execution as P_0, P_1, P_2, P_0, P_1.

```
process P0:          process P1:          process P2:
while (true) {       wait(S1);            wait(S2);
  wait(S0);          signal(S0);          signal(S0);
  printf("0");
  signal(S1);
  signal(S2);
```

Initial	S0=1	S1=0	S2=0	Remark.
P_0	0	1	1	0
P_1	1	0	1	—
P_2	1	0	0	—
P_0	0	1	1	00
P_1	1	0	1	—

2. Consider two processes A & B. Two semaphore variables S and T are used to synchronize the processes. S is initialized to 0 & T is initialized to 1. Processes are scheduled in the following order: A B A B B A A B A. What will be printed on the screen?

```
process A:           process B:
while(1)             while(1) {
{  wait(S);            wait(T);
   print 'P';           print 'I';
   print 'P';           print 'I';
   signal(T);           signal(S);
```

Initial	S=0	T=1	Remarks.
A	0	1	process A is blocked.
B	1	0	II
A	0	1	IIPP
B	1	0	IIPPII
B	1	0	process B is blocked
A	0	1	IIPPIIPP
A	0	1	process A is blocked
B	1	0	IIPPIIPPII
A	0	1	IIPPIIPPIIPP

Write a C program to simulate:

a) Producer - consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int *buffer;
int Buffer_size;
int in=0, out=0;
int total_items;

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg){
    for(int i=0; i<total_items; i++){
        int item = i+1;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("[Producer] Produced: %d -> Buffer[%d]\n", item, in);

        in = (in+1) % Buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}

void* consumer(void* arg){
    for(int i=0; i<total_items; i++){
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("  [Consumer] Consumed: %d <- Buffer[%d]\n", item, out);
        out = (out+1) % Buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
    return NULL;
}

int main(){
    pthread_t p, c;
    printf("Enter buffer size: ");
    scanf("%d", &Buffer_size);
    printf("Enter number of items: ");
    scanf("%d", &total_items);
    buffer = (int*) malloc(sizeof(int) * Buffer_size);
    sem_init(&empty, 0, Buffer_size);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    free(buffer);
    return 0;
}
```

Output:

Enter buffer size : 3
Enter number of items : 10
[Producer] Produced : 1 → Buffer[0]
[consumer] consumed : 1 ← Buffer[0]
[Producer] Produced : 2 → Buffer[1]
[Producer] Produced : 3 → Buffer[2]
[consumer] consumed : 2 ← Buffer[1]
[Producer] Produced : 4 → Buffer[0]
[consumer] consumed : 3 ← Buffer[2]
[Producer] Produced : 5 → Buffer[1]
[Producer] Produced : 6 → Buffer[2]
[consumer] consumed : 4 ← Buffer[0]
[Producer] Produced : 7 → Buffer[0]
[consumer] consumed : 5 ← Buffer[1]
[Producer] Produced : 8 → Buffer[1]
[consumer] Consumed : 6 ← Buffer[2]
[Producer] Produced : 9 → Buffer[2]
[consumer] consumed : 7 ← Buffer[0]
[Producer] Produced : 10 → Buffer[0]
[consume] consumed : 8 ← Buffer[1]
[consumer] consumed : 9 ← Buffer[2]
[consumer] consumed : 10 ← Buffer[0]

		in	out	empty	full
0	[_, _, _]	0	0	3	0
1	[1, _, _]	1	0	2	1
2	[_, _, _]	1	1	3	0
3	[_, 2, _]	2	1	2	1
4	[_, 2, 3]	0	1	1	2
5	[_, _, 3]	0	2	2	1
6	[4, _, 3]	1	2	1	2
7	[4, _, _]	1	0	2	1
8	[4, 5, _]	2	0	1	2
9	[_, 5, _]	2	1	2	1
10	[_, _, _]	2	2	3	0

b) Dining - philosopher's problem.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
sem_t *chopstick;
sem_t room;
int N;
void * dine(void *arg){
    int id = *(int *)arg;
    while(1){
        printf("Philosopher %d is thinking\n", id);
        sleep(1);
        sem_wait(&room)
        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id+1) % N]);
        sem_post(room);
    }
}

int main(){
    int i;
    printf("Enter number of philosophers: ");
    scanf("%d", &N);
    pthread_t philosophers[N];
    int ids[N];
    chopstick = (sem_t *) malloc(N * sizeof
                                 (sem_t));
    sem_init(&room, 0, N-1);
    for(i=0; i<N; i++){
        sem_init(&chopstick[i], 0, 1);
    }
    for(i=0; i<N; i++){
        ids[i] = i;
        pthread_create(&philosopher[i],
                       NULL, dine, &ids[i]);
    }
    for(i=0; i<N; i++){
        pthread_join(philosopher[i],
                     NULL); }
    return 0;
}
```

Output:

Step	P0	P1	P2	P3	P4
1	1	1	1	1	1
2	0	1	0	1	1
3	1	0	1	0	1
4	0	1	1	1	0
5	1	0	0	1	1
6	1	1	1	0	0
7	0	1	0	1	1
8	1	0	1	0	1

Output:

Enter number of philosophers: (max 10): 3
Philosopher 1 is thinking
Philosopher 0 is thinking
Philosopher 2 is thinking
Philosopher 1 is eating
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 2 is eating
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 1 is eating
Philosopher 1 finish eating
Philosopher 1 is thinking
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 2 is eating
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 1 is eating
Philosopher 1 finished eating
Philosopher 2 is eating
Philosopher 2 finished eating

Bad Banker's Algorithm

Safety Algorithm

1. Let Work and Finish be vectors of ~~leg~~ length m and n respectively. Initialize:
   Work = Available
   Finish[i] = false for i = 0, 1, ... n-1
2. Find an i such that both:
   (a) Finish[i] = false
   (b) Need$_i$ ≤ Work
   If no such i exists, go to step 4
3. Work = Work + Allocation$_i$
   Finish[i] = true
   go to step 2.
4. If Finish[i] == true for all i, then the systems is in a safe state.

Q. Consider the max member of resources available for A = 10, B = 5, C = 7.

	Allocation	Max	available	Need
	A B C	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1
	7 2 5			

Need = Max - Allocation

10 5 7
- 7 2 5
3 3 2 → available

P_0 Finish [0] = false

Need ≤ available

743 ≤ 332

P_1 Finish [1] = false

122 ≤ 532

work = work + allocation

= 332 + 200

= 532

finish [1] = true.

P_2 finish [2] = false

600 ≤ 532

P_3 011 ≤ 532

work = work + all

= 211 + 532

= 743

finish [3] = true

P4 Finish [4] = false

431 ≤ 743

work = 743 + 002 = 745

Finish [4] = true

P_0 Finish [0] = false

743 ≤ 745

work = 745 + 010 = 755

finish [0] = true

P_2 Finish [2] = false.

~~work = 755 + 302~~ 600 ≤ 755

work = 755 + 302 = 10 5 7

finish [P_2] = true

the safe sequence is : {P_1, P_3, P_4, P_0, P_2}

{P_1, P_3, P_4, P_0, P_2}

Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
#define MAX 10
int main () {
    int n, m;
    int alloc [MAX] [MAX], max [MAX] [MAX], need[MAX][MAX];
    int total [MAX], avail [MAX], work[MAX];
    int finish [MAX] = {0};
    int safeseq [MAX];
    printf("Enter number of processes:");
    scanf("%d", &n);
    printf("Enter number of resource type:");
    scanf("%d", &m);
    printf("\n Enter Allocation matrix:\n");
    for (int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            scanf("%d", &alloc [i] [j]);
        }
    }
    printf("\n Enter Maximum matrix:\n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &max [i][j]);
        }
    }
    printf("\n Enter total Resources:\n");
    for(int j=0; j<m; j++) {
        scanf("%d", &total [j]);
    }
```

```
for (int j=0; j<m; j++) {
    int sum=0;
    for (int i=0; i<n; i++) {
        sum += alloc[i][j];
    }
    avail[j] = total[j] - sum;
    work[j] = avail[j];
}
printf("Need Matrix:");
for (int i=0; i<n; i++) {
    for (int j=0; j<m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
        printf("%d", need[i][j]);
    }
}
int count=0;
while (count<n) {
    int found=0;
    for (int i=0; i<n; i++) {
        if (finish[i]==0) {
            int j;
            for (j=0; j<m; j++)
                if (need[i][j] > work[j])
                    break;
            if (j==m) {
                for (int k=0; k<m; k++)
                    work[k] += alloc[i][k];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if (found==0) {
        printf("\nSystem is NOT in a Safe state");
        return 0;
    }
}
printf("Available Resources:");
for (int j=0; j<m; j++)
    printf("%d", avail[j]);
printf("System is in Safe state\nSafe Sequence:");
for (int i=0; i<n; i++)
    printf("P%d", safeSeq[i]);
printf("\n");
return 0;
}
```

~~Enter no~~

~~Output~~:

```
Enter number of processes: 5
Enter number of resource types: 4
Enter Allocation Matrix:
0 0 1 2
1 0 0 0
1 3 5 4
0 6 3 2
0 0 1 4

Enter Maximum Matrix:
0 0 1 2
1 7 5 0
2 3 5 6
```

0 6 5 2
0 6 5 6

Enter total Resources:
3 14 12 12

Need Matrix:
0 0 0 0
0 7 5 0
1 0 0 2
0 0 2 0
0 6 4 2

Available Resources: 1 5 2 0
System is in Safe State
Safe Sequence: P0 P2 P3 P4 P1

2 8/4/24

Resource Request Algorithm

```
int p;
printf("Enter process no making request");
scanf("%d", &p);
int request[m];
printf("Enter request vector:");
for(int i=0; i<m; i++){
        scanf("%d", request[i]);}
for(int i=0; i<m; i++){
    if(request[i] > need[p][i]){
        printf("ERROR: request exceeds need\n");
        return 0;
    }
}
for(i=0; i<m; i++){
    if(request[i] > avail[i]){
        printf("Resource not available process
            must wait\n");
        return 0;
    }
}
for(i=0; i<m; i++){
    avail[i] -= request[i];
    alloc[p][i] += request[i];
    need[p][i] -= request[i];
}
int finish[n], safesequence[n];
for(i=0; i<n; i++){
        finish[i] = 0;
}
```

Output:

```
Enter no of processes: 5
Enter no of resources: 3
Enter allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter available resources:
3 3 2
Enter process no making request: 1
Enter request vector:
1 0 2

System is in safe state
Safe sequence: P1 P3 P4 P0 P2
```

Deadlock Detection.

```
#include <stdio.h>
# define MAX 10
int main() {
  ~~int main() {~~
  int n, m;
  int alloc[MAX][MAX], request[MAX][MAX];
  int avail[MAX], work[MAX];
  int finish[MAX] = {0};
  printf("Enter number of processes:");
  scanf("%d", &n);
  printf("Enter number of resource types:");
  scanf("%d", &m);
  printf("\nEnter allocation matrix:\n");
  for (int i=0; i<n; i++)
    for (int j=0; j<m; j++)
      scanf("%d", &alloc[i][j]);
  printf("\nEnter Request Matrix:\n");
    for (int i=0; i<n; i++)
      for (int j=0; j<m; j++)
        scanf("%d", &request[i][j]);
  printf("Enter Available Resources:\n");
  for (int j=0; j<m; j++) {
    scanf("%d", &avail[j]);
    work[j] = avail[j];
  }
  for (int i=0; i<n; i++) {
    int flag = 0;
    for (int j=0; j<m; j++) {
      if (alloc[i][j] != 0) {
        flag = 1;
        break;
      }
    }
```

```
        if (flag == 0)
            finish[i] = 1;
        else
            finish[i] = 0;
    }
    int found;
    do {
        found = 0;
        for (int i = 0; i < n; i++) {
            if (finish[i] == 0) {
                int j;
                for (j = 0; j < m; j++) {
                    if (request[i][j] > work[j])
                        break;
                }
                if (j == m) {
                    for (int k = 0; k < m; k++)
                        work[k] += alloc[i][k];
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
    } while (found);
    int deadlock = 0;
    printf("\nDeadlocked Processes:");
    for (int i = 0; i < n; i++) {
        if (finish[i] == 0) {
            printf("P-%d ", i);
            deadlock = 1;
        }
    }
    if (deadlock == 0)
        printf("None (No Deadlock)");
    printf("\n");
    return 0;
}
```

Output

```
Enter number of processes: 3
Enter number of resource types: 3
Enter Allocation matrix:
0 1 0
2 0 0
3 0 3
Enter Request matrix
0 0 0
2 0 2
0 0 1
Enter Available Resources:
0 0 0
Deadlocked Processes: P1 P2
```

5/5/26

Memory Allocation

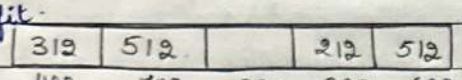
1. Given a free memory partition 400K, 700K, 200K, 300K, 600K. How would each of first fit, best fit, and worst fit algorithms work to place processes of 212K, 512K, 312K, 526K (in order)

First fit

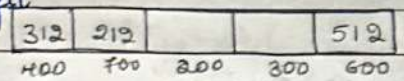
$P_1 = 212k$
$P_2 = 512k$
$P_3 = 312k$
$P_4 = 526k$

526k cannot be allocated with memory.

best fit:



worst fit

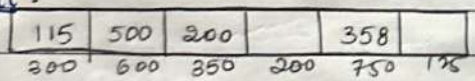


$P_1 = 312$
$P_2 = 512$
$P_3 = 312$
$P_4 = 526$

526 cannot be allocated with memory.

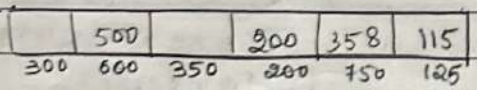
2. Memory partition: 300KB, 600, 350, 200, 750, 125
Process size: 115, 500, 358, 200, 375.

first fit:



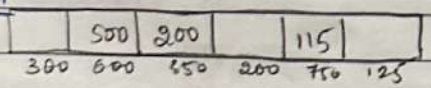
375KB cannot be allocated with memory

best fit:



375 cannot be allocated

worst fit



375, 358 cannot be allocated.

Write a C program to simulate the contiguous memory allocation techniques. a) Worst-fit b) Best fit c) First-fit.

```
#include<stdio.h>
#define MAX 100
void firstfit(int blocks[], int m, int processes[], int n);
void bestfit(int blocks[], int m, int processes[], int n);
void worstfit(int blocks[], int m, int processes[], int n);

void firstfit(int block[], int m, int processes[], int n){
    int allocation[MAX];
    for(int i=0; i<n; i++)
        allocation[i]= -1;
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            if(blocks[j] >= processes[i]){
                allocation[i] = j;
                blocks[j] -= processes[i];
                break;
            }
        }
    }
    printf("First fit Allocation");
    printf("Process No.\t Process Size \t Block No.\n");
    for(int i=0; i<n; i++){
        printf("%d\t \t%d \t\t", i+1, processes[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i] +1);
        else
            printf("Not Allocation\n");
    }
}
```